

DBMS

- collection of logically interrelated data is known as database.

→ Data → Data is basically ~~raw~~ facts or information of an individual identity.

→ Database → Database is collection of logically interrelated data.

→ Database Management → It is the creation & maintenance of a collection of organised data.

→ Database Management system → It is a software package designed to define, manipulate, retrieve and manage data in a database.

→ Need for DBMS :-

(i) It provides a highly efficient method for handling large amount of different types of data with ease.

(ii) Database allows data to be stored systematically.

(iii) Data can be ~~as~~ easily retrieved, filtered, sorted, and updated efficiently and accurately.

→ Components of DBMS :-

(1) Database Users →

(a) End users :- They use the data for queries, reports and some of them

- update the database content.

- End users can be categorised into :-

(1) Sophisticated

(2) Naive

1- Sophisticated users → These include business analyst, scientists, engineers and others who are thoroughly familiar with the system capability.

2- Naive or parametric users → They use pre-defined functions against the database.
eg - Bank tellers for reservation clerks who do this activity for an entire shift of operation using standard types of queries.

(b) Database Administrators :- Responsible for authorising access to the database for coordinating and monitoring its use, acquiring software and hardware resources, controlling its use and monitoring efficiency of operations.

(c) Application Programmers (Software Engineers) :- They are responsible for writing database application programs in some programming language.

Date: [] [] [] [] [] []
Page: [] []

(ii) Data → The collection of facts stored in the database.

- Data is stored, updated and retrieved to form a database.
- Database contain both operational data and meta data.

(iii) Software →

- (a) operating system software
- (b) DBMS software.
- (c) Network software
- (d) Application programs & utility software.
- (e) Procedures

Procedures are instructions and rules that govern the design and use of database system.

(iv) Hardware →

- (a) PC
- (b) Network of computers
- (c) Various storage devices
- (d) Input devices
- (e) Output devices

(v) Database languages →

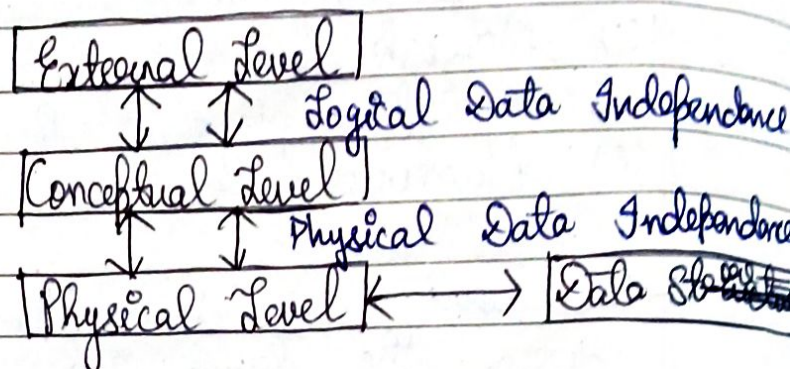
- (a) Data definition language (DDL)
- (b) Data Manipulation language (DML)
- (c) Data Control language (DCL)
- (d) Transaction control language (TCL)

DBMS Architecture

DBMS has 3-tier architecture :-

Level 1
Level 2
Level 3

which divides the complete system into 3 inter related but independent modules as shown below :-



① Physical level → At the physical level, the information about the location of database objects in the store area is kept.

• Various users of DBMS are unaware of the locations of these objects. (like end users)

② Conceptual level ^(design) → At conceptual level, data is represented in the form of various database tables.

For eg → Student database may contain student and course tables which will be visible to users but users are unaware of their storage.

Also referred as logical schema as it describes what kind of data is to be stored in the database.

③ External ^(view) level → An external level specifies a view of the data in terms of conceptual level tables.

- Each external level view is used to cater to the needs of a particular category of users. For eg → faculty of a university is interested in looking course details of students and students are interested in looking at all the details related to academics, accounts, courses & hostel details as well. So different views can be generated for different users.

- The main focus of external level is to provide the view of data.

Data Independence →

○ Data independence means change of data at one level should not affect another level.

- There are 2 types of data independence:-

- ① Physical data independence
- ② Logical data independence.

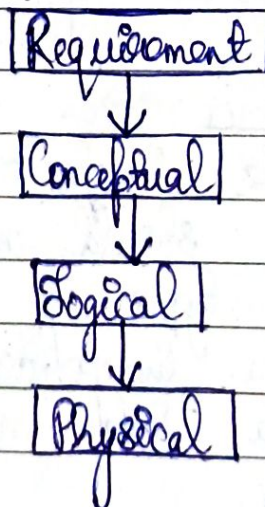
① Physical Data Independence → Any change in the physical location of table and indexes should not affect the conceptual level or external view of data.

- This data Independence is easy to achieve & implemented by most of the DBMS.

② Logical Data Independence → The data at conceptual level schema & external level schema must be independent. This means a change in conceptual schema should not affect external schema.

eg. → adding or deleting attributes of a table should not affect the users view of the table. but this type of independence is difficult to achieve as compare to physical data independence bcoz the changes in conceptual schema are reflected in the users view.

Phases of Database Design



Database designing for a real world application starts from capturing the requirements to physical implementation using DBMS software which consists of following steps:-

Tuples $\rightarrow$ rows
single cell $\rightarrow$ attribute

Date
Page Method

- ① Conceptual design $\rightarrow$ The requirements of database are captured using high level conceptual data model. For eg $\rightarrow$ the ER model is used for the conceptual design of the database.
- ② Logical design $\rightarrow$ Logical design represents data in the form of relational model. ER diagram produce in the conceptual design is used to convert the data into the relational model.
- ③ Physical design $\rightarrow$ In physical design, data in relational model is implemented using commercial DBMS like Oracle, DB2.

Advantages of DBMS :-

- (i) DBMS helps in efficient organisation of data in database which has following advantages over typical file system.
 - $\rightarrow$ DBMS is better than normal file system:-
 - Minimize redundancy and data inconsistency
eg. $\rightarrow$ {multiple times entry} (redundancy).
 $\rightarrow$ {after updation also, data is not proper} (inconsistency).
- (ii) Simplified data access: (user only need table name to access the data).
- (iii) Multiple data views.
- (iv) Data security.

(v) Concurrent access to data.

(vi) Backup & recovery mechanism.

★ Relational model → Relational Data model represents the database as a collection of relations. A relation is nothing but a table of values.

- Every row in the table represents collection of related data values. These rows in the table denote a real world entity or relationship.
- The table name and column names are helpful to interpret the meaning of values in each row.
- The data are represented as a set of relation. In the relational model, data are stored as tables. However, the physical storage of the data is independent of the way. The data are logically organised.

Important terms

1. Attribute → It represents each column in a table. These are the properties which define a relation.
eg. STUDENT NAME, STUDENT ROLL NO etc.
2. Tables → In the relational model, the relation are saved in the table format. It is stored

along with its entities.

- A table has 2 properties, i.e., rows & columns, where rows represents records and columns represents attributes.
- 3. Tuple $\rightarrow$ It is a single row of a table which contains a single records.
- 4. Relation Schema $\rightarrow$ It represents the name of a relation with its attributes.
- 5. Degree $\rightarrow$ The total number of attributes present in the relation.
- 6. Cardinality $\rightarrow$ It represents total number of rows present in a table.
- 7. Relation Instance $\rightarrow$ It is a finite set of tuple in the RDBMS system. Relation Instance never have duplicate tuple.
- 8. Attribute domain $\rightarrow$ Every attribute have some predefined values and set which is known as attribute domain.
- 9. Null value $\rightarrow$ The value which is not known or unavailable is called as null value. It is represented by blank space.
- 10. Constraints (condition) $\rightarrow$ While designing relational model, we

define some condition which must hold for data present in database.

- These constraints are checked before performing any operations (i.e. insertion, updation and deletion) in database.
- If there is any violation of constraints then operations will fail.

(a) Domain Constraints → These are attribute level constraints.

→ An attribute can only take values which lie inside the domain range.

→ For eg :- if a constraint $\text{age} > 0$ is applied on STUDENT relation. Now, inserting value of age will result in failure.

(b) Key Integrity → Every relation in the database should have at least one set of attributes which defines a tuple uniquely.

→ These sets of attributes is defined as key.

→ For eg :- ROLL NO in student relation is a key.

→ Key must have two properties, that are -

(a) It should be unique for all tuples.

(b) It can't have null values.

(c) Referential Integrity → when one attribute of a relation can only take values from other

attribute of some relation or any other relation.

Unit - 2 (ER Diagram)

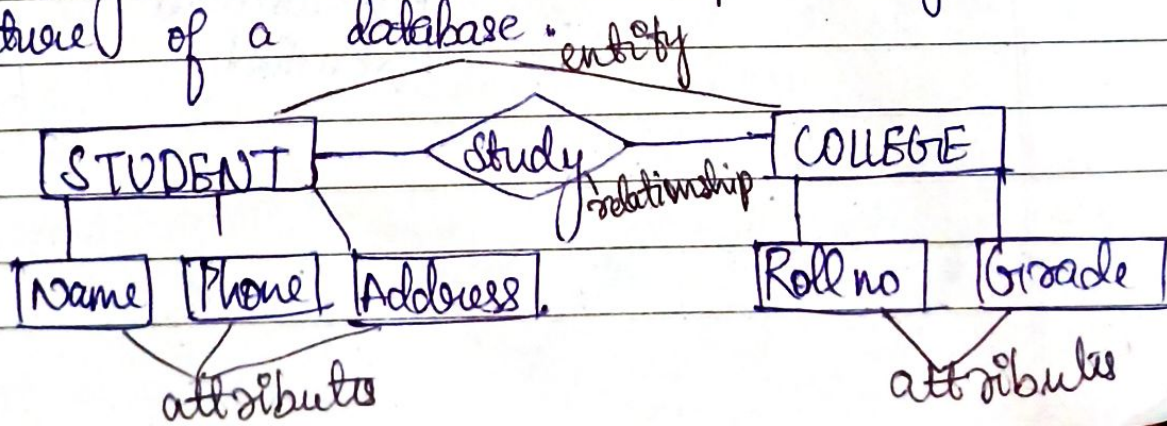
An entity relationship model describes the structure of a database with the help of a diagram which is known as Entity Relationship Diagram (ER Diagram). An ER model is a design or blueprint of a database that can later be implemented as a table data.

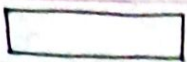
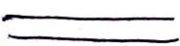
- The main component of ER model are :-

- Entity set
- Relationship set.

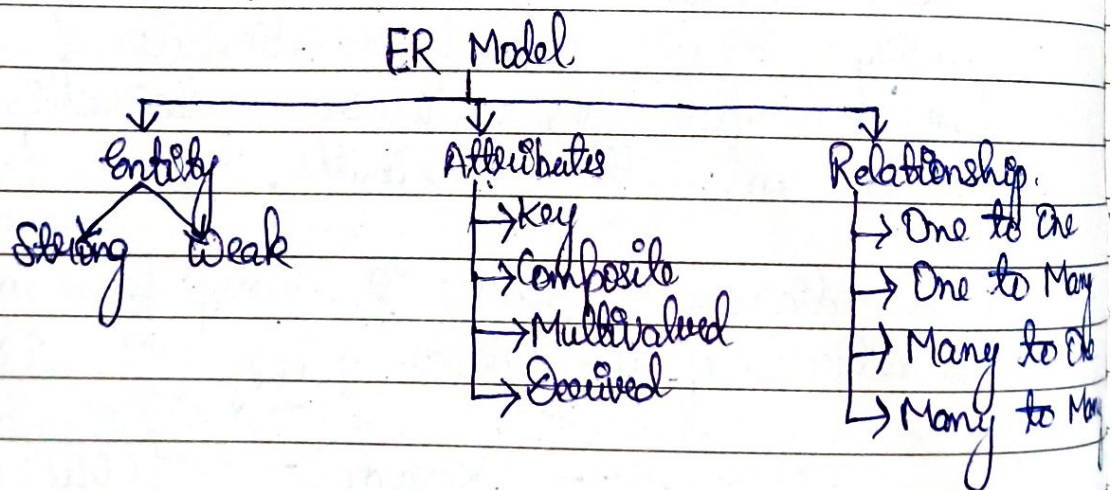
- An ER diagram shows the relationship among entity set. An entity set is a group of similar entities and these entities can have attributes. In terms of DBMS, an entity is a table or attribute of a table in database by showing relationship among tables and their attributes.

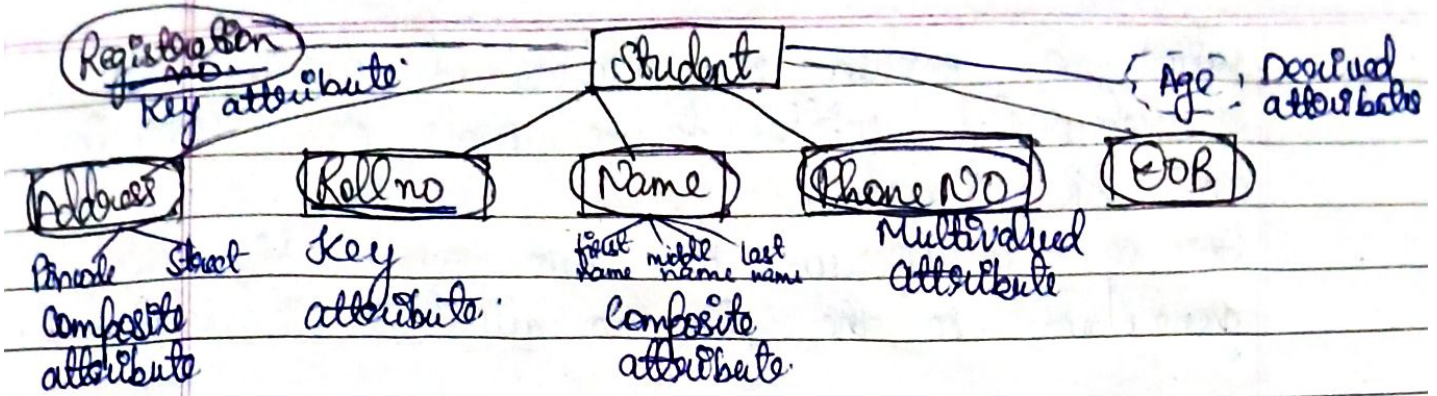
- ER diagram shows the complete logical structure of a database.



- Date:
Page: Notebook
- Rectangle  : Entity
 - Ellipses  : attributes
 - Diamond  : relationships
 - Lines  : Link between
 - entity to attributes
 - entity to relationships
 - Double ellipses  : Multivalued attributes
 - Dashed ellipses  : Derived attributes
 - Double rectangle  : weak entity
 - Double lines  : Total participation of an entity in a relationship set

★ Components of ER model





Attribute → An attribute describes the property of an entity. It is represented as a oval in an ER diagram.

There are 4 types of attributes :-

① Key attribute :-

A key attribute can uniquely identify an entity from an entity set.
 For eg - student rollno can uniquely identify a student from a set of students.

- Key attribute is represented by oval same as other attributes. However, the text of key attribute is underlined.

② Composite attribute :-

An attribute that is a combination of other attributes is known as composite attribute.

For eg - in student entity, student name & address are composite attributes.

③ Multivalued attributes :-

An attribute that can hold multiple

values is known as multivalued attribute. It is represented with double oval or ellipses in an ER diagram.

For eg - a person can have more than one phone no, so the phone no attribute is multivalued.

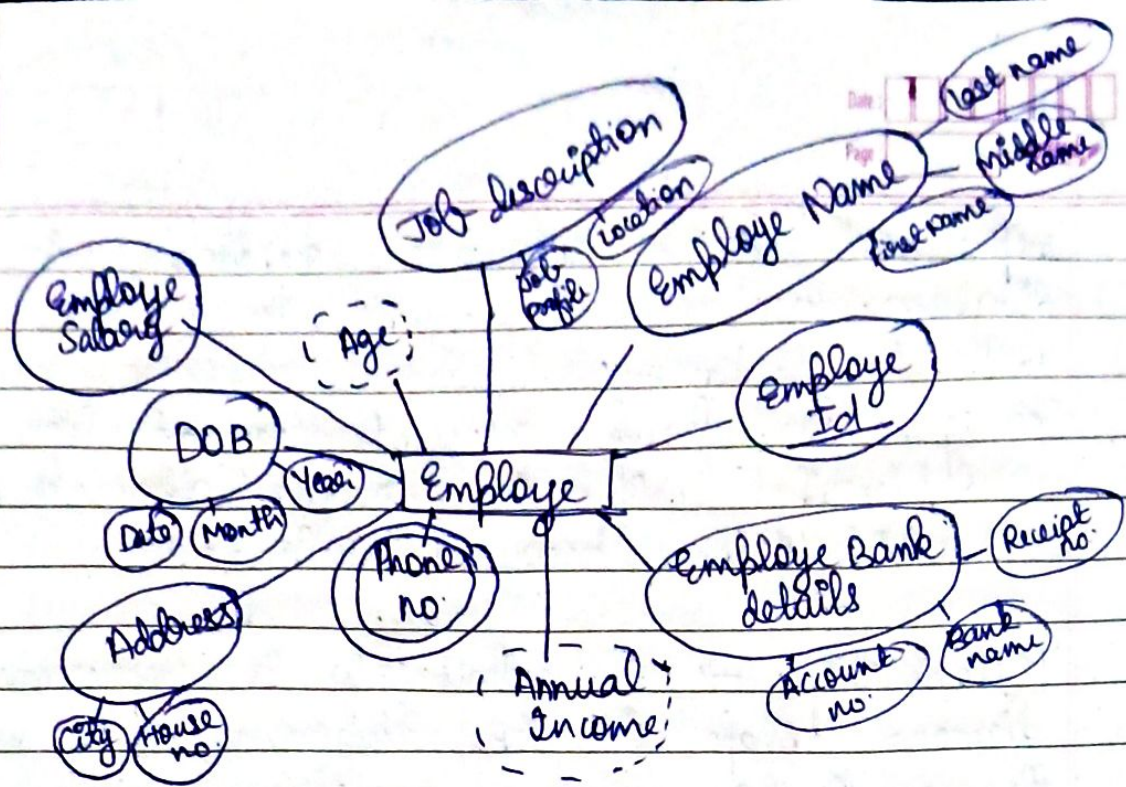
④ Derived attribute :-

A derived attribute is one whose value is dynamic and derive from another attribute. It is represented by dashed oval in an ER diagram.

For eg - person age is a derived attribute as it changes over time & can be derived from another attribute that is DOB.

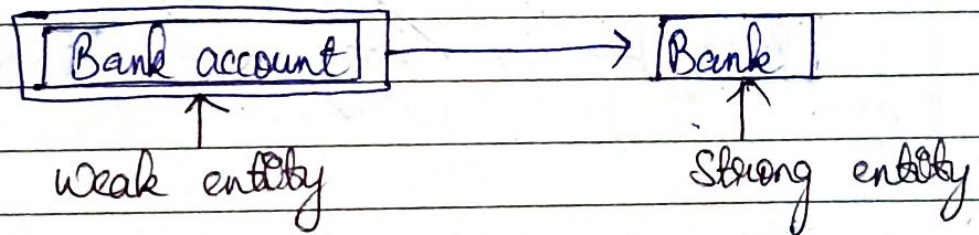
Que. Draw ER diagram for an employee of a company with following attributes:-

- ① Employee - ID
- ② Employee - Name
- ③ Employee - ^{Bank Details} ~~Account no~~
- ④ Address
- ⑤ Employee - Salary
- ⑥ DOB
- ⑦ ~~Age~~ Age
- ⑧ Annual Income
- ⑨ ~~Job~~ Job description
- ⑩ Phone no.



create table Employee
 (Employee_ID varchar(30), Employee_Name varchar(15),
 Employee_BankDetails varchar(30), Address varchar
 (20), Employee_Salary number(5,2), DOB Date, Age
 number(2), Annual_Income Number(8,2), Job
 description varchar(30), Phone-No number);

Entity →



An entity is an object or component of data which is represented by a rectangle in ER diagram.

Weak entity → An entity that cannot be uniquely identified by its own attribute and relies on the relationship with other entity is called as a weak entity. It is

represented by a double rectangle in ER diagram.

For eg - a bank account cannot be uniquely identify without knowing the bank to which the account belongs.

Relationship → A relationship is represented by diamond shape in ER diagram. It shows the relationship among entities. There are four types of relations

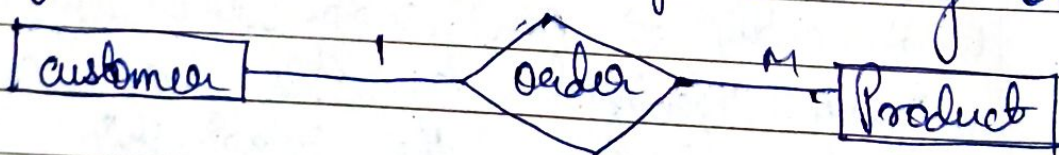
① One to one → when a single instance of an entity is associated with a single instance of another entity, then it is called as one to one relationship.

- For eg - a person has only one passport and a passport is given to one person.



② one to many → when a single instance of an entity is associated with more than one instance of another entity, then it is called as one to many relationship.

- For eg - a customer can place many orders.

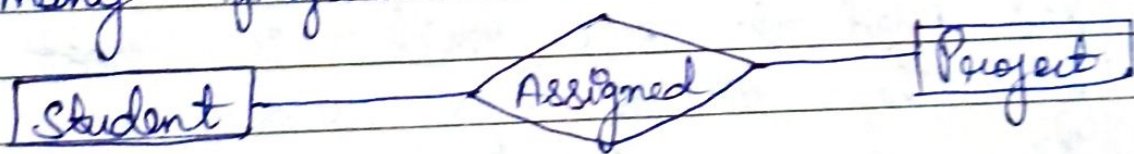


- ③ Many to one → When more than one instance of an entity is associated with a single instance of another entity, then it is called as many to one relation.
- For eg - many students can study in a single college but a student cannot study in many colleges at the same time.



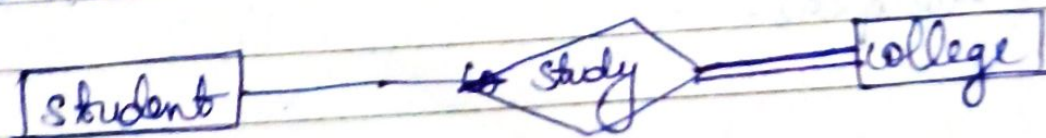
- ④ Many to many → When more than one instance of an entity is associated with more than one instance of another entity, then it is called as many to many relationship.

- For eg - many students can be assigned many projects.



- Total Participation of an entity set requires that each entity in entity set must have at least one relationship in a relationship set.

- For eg - each college must have at least one associated student



→ Keys in DBMS :-

Keys played an important role in the relational database. It is used to uniquely identify any record or row of data from the table. It is also used to establish and identify relationships between tables.

Types of keys :-

- ① Primary key
- ② Candidate key
- ③ Super key
- ④ Foreign key
- ⑤ Alternate key
- ⑥ Composite key

- ① Primary key → It is the first key used to identify one and only instance of an entity uniquely. An entity can contain multiple keys but the key which is most suitable from the list of keys becomes our primary key.

Eg. In the employee table, id can be primary key since it is unique for each employee. In the employee table, we can even select passport no. & license no. as primary key since they are also unique. For each entity key, the primary key selection is based on requirements and developers.

② Candidate key $\rightarrow$ A candidate key is an attribute or a set of attributes that can uniquely identify a tuple. Candidate keys are as strong as primary keys. One of the most relevant candidate key is considered as primary key.

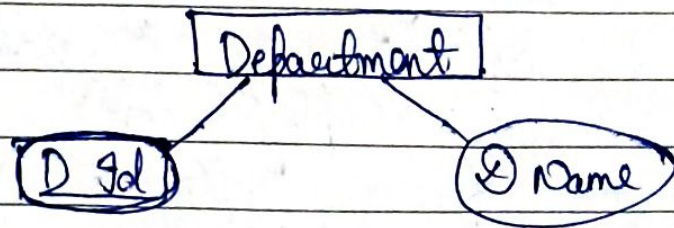
③ Super key $\rightarrow$ Super key is an attribute set that can uniquely identify tuple. A super key is a superset of a candidate key.
Ex. eg. - In the employee table, for (id, name) the name of 2 employees can be same but their id can't be same, so this combination (id, name) is a key.

ID	name
E01	Aman
E02	Aditya
E03	Raj
E04	Raj

Note $\rightarrow$ Primary key can't be null but candidate key and super key can ~~take~~^{take} null values.

④ Foreign key $\rightarrow$ Foreign keys are the column of the table to use to point the primary key of another table.
Every employee works in a specific department in a company & employee and department are two different entity. So we can't store the dept. information in the employee

table, that's why we link these two table through the primary key of one table. So we add the primary key of dept table i.e dept id as a new attribute in the employee table. Now employee table has dept. id has the foreign key and now both the tables are related.



- (5) Alternate key → There may be one or more attributes or a combination of attributes that uniquely identify each tuple in a relation. These attribute of combination of attributes are called as candidate key. One key is chosen as the primary key from these candidate keys and the remaining candidate keys are termed as alternate keys.
- (6) Composite key → Whenever a primary key consists of more than one attribute, it is known as the composite keys. These keys are also known as concatenated keys.

redundancy - repeated for all attributes
information

Unit - 3

Normalization

Database normalization - it is a technique of organizing data in a database. It is a systematic approach of decomposing the table to eliminate data redundancy and undesirable characteristics like insertion, deletion and updation anomalies.

It is a multistep process that puts the data in a tabular form. Normalization is used for mainly 2 purposes:

- ① To eliminate redundant or useless data
- ② Ensuring data dependency make sense - means data should be logically store.

Types of Normalization

- ① 1NF
- ② 2NF
- ③ 3NF
- ④ BCNF (Boyce - Codd Normal form)
- ⑤ 4NF
- ⑥ 5NF

Ques Problems that may face if your data is not normalize:-

→ Insertion, updation, deletion anomalies with eg's of all.

1st Normal form (1NF) →

Rules (or) ^{for} 1NF

① Single-valued attribute which means that each column should contain single attribute.

② Attribute domain should not change.

① eg -

Student-id	Sname	Subject
1	A	C++, Java

X (a)

Student-id	Sname	Subject
1	A	C++
1	A	Java

✓ (b)

The above table is not in 1NF as it is ~~having~~ Subject attribute is not having single attribute for row 1.

For table a to be in 1NF, the correct form is b.

③ Order can vary (means order doesn't matter)

④ Attribute name should be distinct.

Sname	Sname
Riya	Gracy

X
(columns should be distinct)

→ for table to be in 2NF

(i) It should be in 1-NF for sure

(ii) Partial dependency should not be there.

→ All the non-~~key~~ ^{prime} attributes depending upon primary key is called functional dependency.

when non prime attribute depends on one of the primary key is called partial dependency

2nd Normal form

For a table to be in 2nd normal form:-

- (i) it should be in 1NF
- (ii) There should not be any partial dependency.

Partial dependency →

To explain partial dependency, let's first understand what is dependency. We know that primary key is able to uniquely identify all the records in the table, thus every attribute in a table is dependent on primary key, this is also known as functional dependency.

For eg:- In the student table, S id is the primary key and it is able to uniquely identify records in a table.
 on which every other attribute is depend.

Student table

S id	name	reg	branch	str. add
1	A		CSE	
2	B		IT	
3	A		BSC Maths	
4	C			

If we know the S id, we can find out student name, his/her branch etc. However, this is not always the case.

Let's try to extend this eg with two more tables.
One is subject-table and score-table.

Subject-table

Sub-id	Subject-name
1	C++
2	Java
3	Maths

Score-table

Score-id	S-id	Sub-id	marks	teacher
1	1	1	70	C++ teacher
2	2	2	65	Java teacher
3	1	3	77	Maths teacher

Score-table is created to store marks created by the students in the resp. subjects. We are also saving name of the teacher as an additional inf. in the score-table.

In the score-table, S-id is used to know which student's marks are these and Sub-id is used to know for which subject marks are these.

Together, S-id and Sub-id, we form the candidate key. Since it is a combination of 2 keys, it is also known as composite key.

Together they uniquely identify every record in the score-table.

Now let's understand what's a partial dependency :-

In the score-table, there is an attribute called teacher name which is dependent only on one of the two keys of the composite keys combination.

For eg. teacher name is dependent on the subject rather than the student, this is known as partial dependency.

Thus when a non-prime attribute depends ^{partially} on the primary key, this is known as partial dependency.

For a table to be in 2nd N.F, it is imp. that we decompose the score-table & take this teacher-name column to the subject-table. This is how partial dependency decompose.

Now all the 3 tables are in 1NF and there is no partial dependency in them.

Subject-table

Sub_id	Subject_name	teacher_name
1	C++	C++ teacher
2	Java	Java teacher
3	maths	maths teacher